Gain-Only Evaluation Hides Failures: A Ledger for LLM Sentence Expansion in AAC

Anonymous submission

Abstract

People who use AAC (augmentative and alternative communication) apps pick a few words, and a large language model can now turn them into a full sentence that the app may speak for them, sometimes after the user reviews it. The model can quietly change the meaning, for example by dropping a "not". Fixes are usually reported only by how many such errors they remove. We argue that this hides failures, and propose a *ledger* that records, beside each gain, what the fix leaves broken, what it breaks that was fine, what it costs and where it applies, compared with a free baseline and a pass line set in advance. Our test case is a *guard* that makes the model try again when its sentence contradicts what the user supplied. **Does it work?** On 78 fresh item families it raises the automatic pass rate from 78.79% to 88.62% (+9.83 points, of which deleting an unrequested "please" alone earns +3.47), yet 12.13% of targets still fail. A passing sentence can also be unusable: "Is it okay if I bring a friend?" became "Is [I] bring a friend?" and still counted as correct. In a small pilot, people rated repaired sentences usable 58.8% of the time against 89.5% for sentences that never needed repair (20+20 texts, raters not independent: a warning sign, not a proven cost). A second pilot pool showed no gap (77.8% vs. 77.8%). A larger audit of 120 texts with three independent raters each, its decision rule fixed in advance, is in progress. Two other add-ons show the same pattern: a menu of readings loses polite readings, and a one-tap cue passed on development items but missed its pass line on fresh ones. When 20 people typed their own keywords, 20.8% of keyword sets lost the negation before the model saw it. We study stored text, not people, and no AAC user took part; we give a pre-specified design for testing our predictions on people.

1 Introduction

People who use AAC often compose by selecting a few words or symbols, because every selection costs time and effort (Valencia et al. 2023; Cai et al. 2024). LLM keyword expansion turns `want go party` into a full sentence (Shen et al. 2022; Cai et al. 2024), which a system may speak on the user's behalf, sometimes after review (Hancock, Naaman, and Levy 2020; Valencia et al. 2020). But the model decides more than words: whether something happened or did not, whether the sentence asks or tells, whether it says "please". Given `not like medication`, a sentence that drops the negation, turns it into a question or adds politeness is fluent and wrong. Three kinds of fix are being explored: check the output against the user's words and regenerate on a conflict (Madaan et al. 2023; Gou et al. 2024), offer suggestions that span the readings the words leave open, or let the user add a one-tap cue such as `[NOT]` (Valencia et al. 2024; Mao et al. 2025; Xiang et al. 2025). Each is usually judged by how many errors it removes. That hides what a fix leaves unfixed, what it changes that was already right, and whether the repaired text is something a person could use.

User input	Model wrote	Fix returned	What the scorer and the person see
`[?] [I] bring friend`	"Is it okay if I bring a friend?"	"Is [I] bring a friend?"	Scored a *pass* (it keeps the question label). Not a sentence.
`knock come`	"Please knock before you come in."	"Don't knock before you come in."	A menu meant to widen choices turned a polite request into a prohibition.
`[ASK] [YOU] [PLAIN] wear headphones`	"You wear headphones."	"Ask you to wear headphones."	Scored a *pass*. Not what was meant.

Figure 1: Illustrative failures found by the audit, real stored outputs. They are not typical: each is the first case in a fixed hash order within its category, and the rate of its category is given here. Row 1: guard changed 26 of 1,394 texts that were already correct. Row 2: menu rewrite; sets offering both a plain and a polite reading fell from 26/40 to 12/40. Row 3: second of the first two repairs in hash order (the first is a clean fix, Table 3); 4/63 scored repairs begin this way, a post hoc count.

Thesis and ledger. Gain-only evaluation hides failures, so we report a ledger. Beside each benefit it records *offline failure indicators* (events in stored text that a gain-only report omits: failures that remain, correct outputs the fix changed,

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

readings a menu no longer offers), the cost, and where the intervention applies. None of these indicators is a measured harm to a person. Our running case is keyword-to-sentence expansion with four LLMs on minimal-pair item families, defined in Section 3.

Contributions. (1) A ledger and its design rules: attribution decides what an intervention may be credited with; a free baseline and pass lines fixed in advance decide whether it merits a human study (§5). (2) One lead case, a commitment guard, evaluated end to end: a confirmation set with a frozen rule, a modest increment over a free baseline, and a human usability check on its repairs (§6). (3) Two bounded secondary rows, and a bridge to human-written pairs and human-chosen keywords. (4) A pre-specified human-effect design that turns the ledger into falsifiable predictions about people (§7). We ran no study with AAC users; every result is a change in stored text.

2 Related Work

LLMs in AAC. LLM expansion and suggestion systems for AAC include KWickChat (Shen et al. 2022) and SpeakFaster (Cai et al. 2024), and AAC users have described how such systems help and hinder them (Valencia et al. 2023). Several add user-side control over intent (Valencia et al. 2024; Mao et al. 2025; Xiang et al. 2025; Lacosse, Tano, and McNamee 2026); others measure whether affect or intent survives generation (Choudhury, Kumar, and Martin 2025; Gorenshtein et al. 2026). Frisch et al. argue that AI-powered AAC needs technical, user-mediated and functional evaluation together (Frisch et al. 2026); our ledger is a technical layer shaped so that a functional study can test it.

Editing, preservation and evaluation. Generate–critique–revise loops (Madaan et al. 2023; Gou et al. 2024) and conditioning a rewrite on the source dialog act (Qu, Ungar, and Sedoc 2023) are established, as is reporting a targeted change together with collateral change (Ziegenbein, Stahl, and Wachsmuth 2026; Elgaar and Amiri 2025). Style-transfer meta-evaluation shows that preservation metrics look valid only when test data do not confound the intended change with content errors (Pauli, Augenstein, and Assent 2025); detoxifying rewriters default to polite output that distorts intent (Wang et al. 2025), and models have default politeness strategies (Zhao and Hawkins 2025). We claim neither the loop nor gain–harm accounting. We add accounting conditioned on what the input *supplied*, against a free baseline, plus a human usability check on repaired text. Suggested text shifts what people write and believe (Arnold, Chauncey, and Gajos 2020; Jakesch et al. 2023; Williams-Ceci et al. 2026; Hohenstein et al. 2023; Agarwal, Naaman, and Vashistha 2025); those are human-outcome studies of exposure, whereas ours is a pre-deployment study of interventions meant to *prevent* unwanted shifts. LLM judges (Zheng et al. 2023) can agree with each other more than with people on pragmatic labels (Wang, Sun, and Guo 2026; Panickssery, Bowman, and Feng 2024), which bounds what our machine labels can show.

3 Setting

Items and populations. An item is a minimal-pair family ("I don't like the new medication" / "I like the new medication"); each source sentence is shown to the model under five input conditions (Table 1). The *development set* has 58 constructed families on six dimensions (after quarantining two families whose scoring rule has a known blind spot); the guard was built and tested there, so its numbers are development results, and the menu and cue rows use it. The *confirmation set* has 240 fresh constructed families (polarity, speech act, politeness), 238 analyzed after a pre-generation exclusion, with three contrasts pre-specified and git-timestamped. Its confirmation is qualified because 48 cached requests (12 distinct primary ones) predate the final item freeze. Its 78 speech-act families are the separate population on which the guard was rerun unchanged. The *human-written pairs* are 547 kept minimal pairs written by 20 non-author annotators. The *keyword study* has 20 annotators type their own keywords for 600 AACText messages (fictional messages written by crowd workers imagining AAC use, not messages of AAC users). The AAC input studied is word input only; symbol and grid-based AAC are out of scope.

Scoring and attribution. Four fixed models write each sentence: Qwen2.5-7B-Instruct, Llama-3.1-8B-Instruct and Gemma-3-4B-it (4-bit, llama.cpp) and one API model. An LLM extractor (two prompt variants) labels each output's dimensions without seeing the source, model or condition; a second-family arbiter resolves disagreements on critical labels; deterministic code turns labels into verdicts, identically for treated and untreated outputs. A target is one (item, model, input condition) outcome. Because the input condition fixes which values the model was given, a miss is attributed to one of three causes: *loss* (the input never carried the value), *violation* (it carried the value and the output changed it) or *assumption* (the input was ambiguous and the output committed anyway). This holds only as far as the extractor and the conditions are right. A checker can enforce only a value the user supplied, so the guard is credited with violations alone; when the keywords leave a dimension open, the remedies are to show alternatives (menu) or to let the user supply the cue. In the confirmation set, 259 disagreement rows (177 units) were resolved by a single arbiter after outcomes were known, blind to reference, model and condition; masking them as unknown gives the same three conclusions. Against two human annotators on 163 sampled untreated outputs, extractor labels agree at held-out $\kappa = 0.91$–0.97 on all 7 label fields (pre-specified gate 0.6). That check covers untreated outputs only; it does not validate repaired or rewritten text, and the pilot audit (Section 6.2) gives the only human check of repairs. The item construction and scorer are specified in an anonymous concurrent paper (Anonymous 2027); nothing in this paper's argument depends on it beyond the description above.

4 Interventions

The commitment guard (lead; system side). After the first pass, a deterministic lexical monitor (regexes and lexicons for negation, person, modality, question form, numbers,

Glossary			
Family	Two source sentences differing on one critical dimension, with the same bare keywords	*Cue, chip*	One tap that adds a commitment, such as `[NOT]`, `[?]`, `[PLAIN]`
Critical dimension	Polarity, agent, certainty, speech act (ask, tell, request), politeness, time/quantity	*Input conditions*	Keywords only; keywords plus a length-matched neutral word; plus the critical cue; plus all cues; the typed sentence
Strict pass	Keeps the critical value, contradicts no supplied dimension, adds no politeness	*First pass*	The model's single greedy sentence, before any intervention
Joint pass	Keeps the critical value and every other scored supplied predicate	*Strip-only*	Free baseline: delete an unrequested "please", "pls" or "plz" (no model call)
Violation	The input carried a value and the output changed it	*Offline failure indicator*	Event in stored text that a gain-only report omits

Table 1: Glossary. All scores are partial predicates computed from machine-extracted labels, not judgments of usability.

tense and "please") compares the output with the values in the visible input: the cues, or for typed input the monitor's own labels of the typed sentence. It also flags a politeness marker that was not supplied. On a flag, the same conversation receives one feedback turn ("Your sentence must keep: negated; no added politeness. Write the sentence again with the same meaning otherwise.") and is regenerated, up to two retries. The guard returns the first unflagged attempt, else the attempt with fewest flags, ties going to the original. The strip-only baseline covers one failure type (an added "please"); it is a floor for a trivial rule, not a competitor that handles negation, person or sentence type. Until then the increment over strip-only is an upper bound on what the feedback message itself contributes.

The menu (secondary; keywords only). Starting from a baseline menu of three suggestions, the complete menu keeps the top suggestion and, for polarity then certainty, rewrites the third (then second) slot into a missing reading with one model call, checked by the monitor with one feedback retry. It was chosen over a variant on a 12-family pilot before the test run.

The cue (secondary; user side). The user adds one commitment cue, such as `[NOT]`, `[MAYBE]`, `[?]` or `[PLAIN]`. We treat it as a *probe* of whether information the user supplies is used, not as a proposed interface. Its real cost depends on the access method (one dwell in eye gaze, a menu traversal in scanning, a fixed button in a symbol grid), and it adds a vocabulary to learn, a decision for every message, and a risk that cues are copied literally into the output; none of these is measured here.

5 Evaluation Design

Paired comparisons of stored configurations. For system-side interventions, the untreated outcome is the *stored* first-pass output for the same request, and the treated outcome is what the intervention returns from it. This removes between-request variation but does not identify an effect on people. Pairing matters because the same prompt does not give the same text twice: reissuing a request reproduced the text in only 76.4–90.8% of local and 44.4–71.4% of API requests. Estimands are row-weighted means over targets; the four models are fixed, not sampled. Intervals are 95% percentile intervals from whole-family or cluster resamples (2,000 on development, 10,000 elsewhere; fixed seeds); they are pointwise and do not correct for exploratory selection or label error. Thousands of rows are not thousands of independent tests: the guard's 63/80 development repairs come from 26 families.

Offline failure indicators and cost. *Changed-correct* counts already-correct outputs whose text the intervention changed; *correct→failure* is the share that became scored failures; *residual* failures remain after the intervention. We also show what a label cannot see, such as scored passes that are fragments, and, for repaired text, a human usability rating. Cost is extra model calls and latency (local latency is not a deployment prediction). Pass lines are coarse by design: they decide whether an intervention merits a human study.

6 Results

6.1 The guard: most scored failures repaired, a modest increment over a free baseline

On the development cue conditions, strict pass rose from 94.25% to 98.78% of 1,392 targets (Table 3). Of the 80 initial failures the guard repaired 63 (78.8% [68.4, 88.9]), far above its 30% pass line. A gain-only report stops there. Deleting an unrequested "please" at zero calls already reaches 1,346 passes, so regeneration's increment is 2.08 points [1.15, 3.16]: 31 targets pass only with the guard and 2 only with deletion. This compares stored configurations and does not isolate the feedback message. 17 of 80 failures remained, 12 of them involving speech act, and 14/17 were still flagged after both retries: detected, not repaired. The guard changed the text of 49/2,154 already-correct outputs, none into a scored failure; yet "Would you mind repeating the last question" became "Please repeat the last question", still polite by the label and no longer what was typed. It flagged 157/1,856 inputs and added 0.104 model calls per input.

Confirmation. Speech act was the guard's weakest dimension, so we reran the guard unchanged on the confirmation set's speech-act families (78 families, four models, three cue conditions, 1,872 targets). Criteria were frozen before any retry: at least 30% of strict failures repaired, at most 10% of strict-correct texts changed, and a joint-pass gain whose

	Guard (lead)	**Menu (bounded)**	**Cue (bounded)**
Defect targeted	Violation of a supplied value	Assumption under ambiguous input	Loss of a value at input
Comparison	Stored first pass for the same request; strip-only (0 calls)	Stored baseline menu, same request	Length-matched neutral word
Population	Development: 1,392 cue targets; confirmation: 1,872 speech-act targets	Development, keywords-only sets of three	Development: 464 targets; confirmation speech act: 624 targets
Primary benefit	Strict pass; joint pass on confirmation	Sets offering both readings	Critical value kept
Offline failure indicators	Failures that remain; correct text changed; passes that are fragments; *human-rated usability of repairs*	Coverage lost on other dimensions; rewrites changing sentence type	Matches lost; misses unrepaired; gap to typed input
Applicability	Supplied cues only	Ambiguous input only	Dimension-specific
Cost	Model calls, latency	Model calls, latency	Selections (access-dependent)
Pass line (fixed in advance)	Repair $\geq$30% of failures *and* change $\leq$10% of correct outputs; confirmation joint-pass gain with lower bound > 0; deployment clause: usable $\geq$ control -10 points (Section 6.2)	Gain $\geq$30 points over baseline menu	Gain $\geq$10 points *and* $\geq$80% of typed-input rate

Table 2: Evaluation card: intervention, comparison, population, outcomes. Verdicts are in Table 3, Fig. 2 and Section 6.2. Development pass lines were fixed in the analysis code and checked on pilot data before the run, without external timestamps; the confirmation criteria were frozen in a hashed file before any retry request.

lower 95% bound exceeds zero. Joint pass rose from 78.79% to 88.62%, 9.83 points [8.39, 11.32]; the guard repaired 251/478 strict failures (52.51% [48.36, 56.70]), changed 26/1,394 strict-correct texts and turned 0 into scored failures. Every criterion passed within the call caps. Deletion alone gained 3.47 points, so the increment over it is 6.36 points [5.29, 7.48], and 12.13% of targets remain strict failures. Row 1 of Figure 1 shows what a changed-correct text can look like: the question label survives, so the output remains a scored pass.

6.2 What the pass does not show: repair usability

A pass checks a few labels, not a sentence. For `[ASK] [YOU] [PLAIN] wear headphones`, the guard turns "You wear headphones." into "Ask you to wear headphones.", which passes every scored predicate (Figure 1). We therefore had people judge repaired text. In a pilot audit, 14 reimbursed non-author raters judged 20 repaired outputs and 20 unchanged controls (outputs that had never failed) for usability, blind to which was which. By item consensus, repaired texts were usable in 10/17 rated items (58.8%) against 17/19 (89.5%) for controls, a gap of -30.7 points [-54.7, -2.0], with 3 and 1 items unresolved. The raters are not independent, the controls had never failed so repaired and control items differ in difficulty, and the interval is wide: the pilot is a warning sign, not an established cost. A second pool of the same design (confirmation-set outputs) showed no gap: 77.8% against 77.8%. Pooling the two pilot pools, a descriptive post hoc view, gives 68.6% against 83.8%, a gap of -15.2 points [-33.8, 4.5]: it includes both zero and the -10-point line, so it neither shows nor rules out a cost. Raters mostly agreed that the meaning of a scored-passing repair was kept (pilot: 14/16 resolved items; second pool: 16/19), so the problem is more often fluency and form (Table 4) than lost meaning: chip echoes, dropped content words and fragments, all passing the scored predicates. **The decisive test.** A larger audit of 120 new texts (half repaired, half never-failed controls, none from the pilots, 3 independent raters each, plus original-failed and strip-only arms to separate repair from item difficulty) has a decision rule frozen before any return: a pooled interval entirely below -10 points means the guard is killed for deployment and kept as a detector; one that includes -10 but excludes 0 means a cost of uncertain size; one that includes 0 means no detectable cost. Neither audit is selected for being favorable. The audit is still in progress; its result will replace this sentence.

6.3 Secondary rows: the menu and the cue probe

Menu (bounded). Under keywords-only input, baseline menus offered both polarity or both certainty readings in 15/80 sets; the complete menu raised this to 78/80, a paired gain of 78.75 points [70.00, 87.50], passing its pass line. On separate politeness families, sets offering both a plain and a polite reading fell from 26/40 to 12/40 (-35.00 points), and on speech-act families from 14/31 to 6/31 (-25.81 points); gains and losses are on different families, not a within-item trade. For `knock come`, "Please knock before you come in." became "Don't knock before you come in.", removing the polite option (Figure 1). Of 428 rewrites, 138 (32.2%) changed sentence type. Applicability matters: when the input already fixes a value, a menu prompt asking for different meanings departs from it, with a departure in 62.76% of baseline menus against 2.70% of sample sets for typed input. That gap reflects prompt design (one prompt asks for different readings, the other draws samples) and does not separate prompt from temperature; it supports offering alternatives only on dimensions the input leaves open.

Cue probe (bounded). On the development set, one cue raised critical-value match from 41.8% with the length-matched word to 94.8% (97.3% of typed input), passing

	Development 58 families, 6 dimensions, 1,392 cue targets	**Confirmation (frozen)** 78 families, speech act, 1,872 targets
Benefit (% of targets)		
Strict pass: first pass / strip-only / guard	94.25 / 96.70 / 98.78	74.47 / 81.73 / 87.87
Joint pass: same order	94.68 / 96.77 / 98.85	78.79 / 82.26 / 88.62
Guard − first pass, points	+4.53 strict	**+9.83** [8.39, 11.32] joint
Strict failures repaired	63/80 (78.8%)	251/478 (52.51%)
Free baseline: strip-only, zero calls		
Strip-only − first pass, points	+2.44 strict	+3.47 [2.78, 4.22] joint
Guard − strip-only, points	**+2.08** [1.15, 3.16] strict	+6.36 [5.29, 7.48] joint
Pass only with guard / only with strip-only	31 / 2	123 / 4
What remains (offline failure indicators)		
Residual strict failures	17/1,392 (1.2%)	227/1,872 (12.13%)
Correct text changed	14/1,312 cue; 49/2,154 all inputs	26/1,394 (1.87%)
Correct → failure	0/1,312	0/1,394
Cost and verdict		
Extra calls per input	0.104 (1,856 inputs)	0.41 (1,848 inputs)
Pass line	passed	passed (all criteria)

Visible input	First pass	Guard output
Development set, repaired (Gemma, one cue; 1st by hash order; 63/80 scored failures repaired)		
`[PLAIN] let finish sentence`	"Please let me finish my sentence."	"Let me finish the sentence."
Development set, scored repair that is a fragment (Llama, all cues; 2nd by hash order; 4/63 scored repairs begin "Ask you", a post hoc lexical count, not a fluency rate)		
`[ASK] [YOU] [PLAIN] wear headphones`	"You wear headphones."	"Ask you to wear headphones."
Development set, failure that remains (Llama, all cues; 1st by hash order; 17/80 failures remain, 14/17 still flagged; the source asks whether the listener is charging it)		
`[?] [YOU] charge wheelchair`	"Can you charge my wheelchair?"	"Can you charge my wheelchair?"
Confirmation set, correct text changed, still a scored pass (Qwen, all cues; 1st by SHA-256 order; 26/1,394 strict-correct targets changed, none to a scored failure)		
`[?] [I] bring friend`	"Is it okay if I bring a friend?"	"Is [I] bring a friend?"

Table 3: The guard as one display. Left: benefit, the free baseline and what remains, on two *separate* populations that are never pooled; scored partial predicates, not usability. *Targets* are scored (item, model, input condition) outcomes; *inputs* are the distinct requests the guard processed. Development: 1,392 cue targets, plus 464 requests with no supplied cue (no strict target), give 1,856 inputs. Confirmation: 1,872 targets share 1,848 distinct requests. Differences are computed from unrounded rates, so they can differ by 0.01 from the difference of the printed rates. Right: stored examples, verbatim, each following the rule named in its row; they are not typical. Brackets: 95% whole-family bootstrap intervals for paired differences. Repair rates condition on initial strict failures; changed-correct and correct→failure on initial strict passes.

both clauses. On 78 fresh confirmation-set speech-act families, a contrast pre-specified and git-timestamped, it raised preservation from 41.83% to 73.24%, 31.41 points [25.00, 37.94], so the gain clause replicates. The $\geq$80% clause does *not*: typed input preserved speech act in 99.20%, so one cue reaches 73.8% of it. The cue passed its development test and failed one clause of its fresh-item test. Question cues fail most visibly: 58/208 question targets became non-questions, as when `[?] parcel signature` became "Can you sign for this parcel?". Cues can also interact: a single local model lost an already-kept negation in 55/640 cases where the cues `[NOT]` and `[YOU]` appeared together (other combinations 0/480), which argues for per-model regression tests of cue combinations. Among the 194 targets kept by the neutral word, the cue lost 7; among the 270 it missed, 17 stayed missed.

6.4 Bridge to realistic input

Human-written pairs. Constructed families may not look like what people write, so non-author annotators wrote 547 kept minimal pairs, run through the same pipeline with the same four models. The cue gain over the length-matched word replicates on all 6 dimensions (35.3 to 82.7 points); the three confirmation contrasts replicate in direction: speech-act cue +35.3 points [31.2, 40.3] (confirmation set +31.41), politeness addition +7.6 [5.0, 10.4] (+6.25), and a bare affirmative default 99.8% [99.3, 100.0]. This replication is exploratory and not pre-specified, each contrast rests on only 6–7 writers (so we claim direction and consistency, not interval width), and on binary dimensions the design caps the gain near 50 points; sensitivity checks moved no estimate by more than 4.4 points.

Human-chosen keywords. In the keyword study, 20 annotators typed keywords for 600 AACText messages (1800 sets). About one keyword set in five lost the negation:

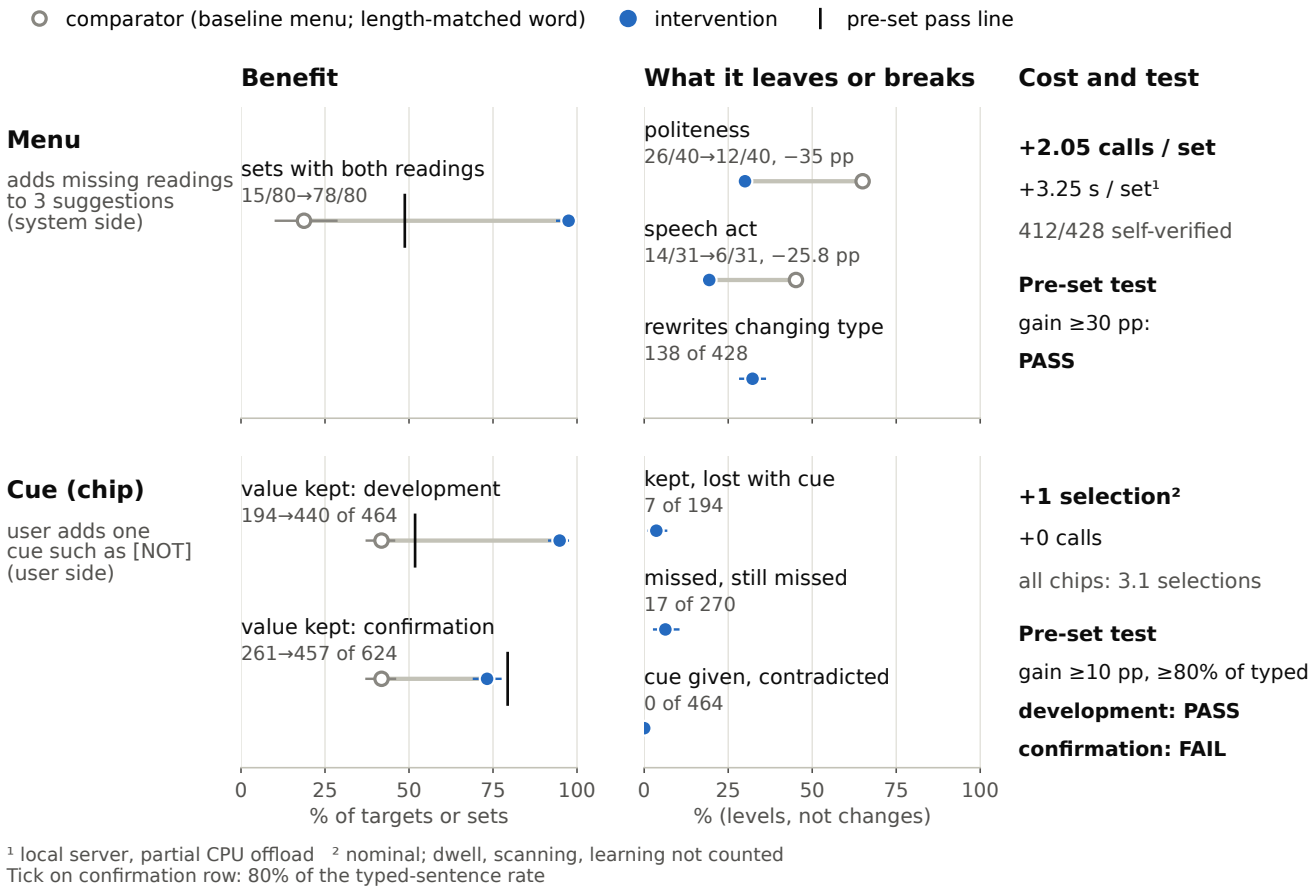


Figure 2: Two add-ons help in one place and cost something elsewhere (machine-extracted labels; four models). Open circle: comparator; filled: intervention; bars: 95% cluster-bootstrap intervals; black tick: the pass line set in advance; pp: percentage points. Plotted values are percent *levels*; changes in points are printed. The menu's politeness and speech-act rows come from families different from its target gain (40 and 31 sets). The cue passes both clauses on the development set but *fails* its ≥80%-of-typed clause on the confirmation set (only its gain clause replicates). The guard is in Table 3.

20.8% [16.9, 25.2]. A rule-based four-word compressor lost it 100.0% of the time; that comparison only shows that human keywording is not that crude rule, and it does not say how often real AAC users drop negation. Politeness reverses: people lost the polite cue more often (35.7%) than the compressor (17.3%), and raters disagree on what keywords carry (Krippendorff $\alpha = 0.61$).

7 From Ledger to a Pre-Specified Human-Effect Study

The ledger is a pre-deployment screen, useful only if it predicts human outcomes. We state falsifiable predictions, the study that tests them, and what would demote each. Everything here is pre-specified and git-timestamped but unrun with AAC users.

Predictions. H1 (writer intent). Among outputs the scorer counts as preserving a writer's critical value, writers report that the output "said what I meant" more often than among outputs it counts as failing; a gap whose interval includes zero demotes the scored endpoint. The first test is the *writer-intent check*: each of the 20 human-pair writers judges 10 outputs generated from their own pairs (five with keywords only, five with one cue), choosing between their two source readings, "something else" or "unclear". Its result is not yet in. **H2 (repair usability).** A scored repair is usable at no less than 10 points below a never-failed control; the 120-text audit tests this under the frozen rule of Section 6.2. **H3 (collateral).** Offline changed-correct and coverage loss predict a later drop in users' "said what I meant" rate; if not, evaluation of this kind should be demoted. H1 and H2 are testable now on writers and raters; H3 needs AAC users.

Study for H3: a micro-randomized trial. *Estimand.* For adult AAC users who compose with LLM keyword expansion, the average causal excursion effect (Boruvka et al. 2018) on a proximal outcome of delivering the regenerated sentence versus the original, at decision points where the monitor flags a conflict, averaged over messages and time within user. *Arms and unit.* Randomization is per flagged message within user (Klasnja et al. 2015), with fixed probabilities,

Pool	Visible input	Stored repaired output
Pilot	`[ASK] [YOU] [PLAIN] speak slowly`	"Speak slowly to you."
	`[ASK] [YOU] [PLAIN] charge wheelchair`	"Ask you for the wheelchair charge."
	`evening take pills`	"Take pills evening."
Second	`[ASK] [YOU] [PLAIN] email manager`	"Ask you email manager."
	`[ASK] turn conditioner`	"Give me a turn conditioner."
	`[TELL] text plumber`	"The plumber."

Table 4: Repairs raters judged unusable by consensus, verbatim. Rule: the first three consensus-unusable repaired items in frozen hash order in each pool (pilot: 7/17 of rated repairs unusable; second pool: 4/18). A cue echo (a repair that begins "Ask you" or "Tell you") opens 1 of the three pilot examples and 1 of the three second-pool examples (3 and 1 of all unusable repairs in the pools).

among three arms: silent regeneration, flag-and-confirm (the user sees the flag and chooses), and the original. The guard's detection value motivates the confirm arm. Cues can only be offered, an encouragement design with its own estimand, so they enter as a separate between-user factor, not randomized per message. *Outcomes.* Proximal: sent without edit, one-tap "said what I meant" on a random subset of sends (not every send, to limit burden), time to send. Longitudinal: weekly rates of both, abandonment of the feature, and reliance (whether acceptance of flagged text rises over weeks), with week as a pre-specified moderator. *Sample-size reasoning.* The monitor flags about 8.5% of inputs, so only a small share of messages are randomized: 40 randomized decision points per user need roughly 473 messages, which is weeks of use. Assuming a standardized proximal effect of 0.2, within-user correlation 0.3 and 80% power at the 5% level (typed assumptions, to be replaced by estimates from a first small pilot), about 125 users are needed; we make no claim to be powered before that pilot. *Analysis.* Weighted and centered least squares on the pre-specified proximal outcome with user-clustered errors; moderators and the longitudinal contrasts are reported with intervals and without selection. *What the human audits can and cannot do.* The audit's 60 versus 60 texts can detect a gap of about 16 points when the control rate is near 89%, so a finding of no detectable cost is not evidence of no cost; the writer-intent check has 200 judgments, an effective 71 at an assumed within-writer correlation of 0.2, enough to detect a gap of about 23 points in the worst case. Both can demote the ledger's endpoints; neither stands in for the trial.

8 Limitations

No human outcomes. Nothing here measures communication success, agency, effort or trust; offline failure indicators are not experienced harms. *Repair audit.* The pilot has 20 + 20 items and non-independent raters, and its controls never failed; the larger audit is the primary estimate and is not yet in. *Machine labels.* Endpoints use an LLM extractor and deterministic verdicts, validated by two human annotators on untreated outputs only; guard repairs, menu rewrites and new boundary cases are not covered, and shared monitor and extractor errors can inflate repairs. *Baseline.* The only free baseline deletes an unrequested "please", one failure type; a resample baseline without feedback text is not in these results. *Scope.* The guard has two separate populations and was developed on the development set; the menu was run on neither the confirmation set nor supplied input; the confirmation set's status is qualified by early cached requests and post-outcome arbitration; the applicability contrast and the cue-combination probe are exploratory; the human-written pairs are exploratory with few writers. *Constructed items, fixed models.* Four fixed models describe these systems, not other models; AACText is crowd-written fiction, and word input is the only modality studied. *Authorship and selection.* Many AAC users compose by rewriting or choosing among options, which we do not model; examples follow declared, outcome-conditioned rules and illustrate rather than sample. *Community involvement.* No AAC user or autistic adviser contributed to the design or analysis; we draw on guidance such as AASPIRE (Nicolaidis et al. 2019) and on the double-empathy (Milton 2012) and heterogeneity (Zisk and Dalton 2019) cautions, assume no "autistic style", and limit claims to commitments explicit in the text. Any claim about benefit requires direct involvement of AAC users and advisers. *Ethics.* No data were collected from AAC users. All annotators were adult non-authors recruited from the authors' university peers, identified only by an ID; each consented and was reimbursed for their time. They labelled model outputs and audited items under a fixed codebook, blinded to model, condition and source; some also chose keywords for public AACText messages or wrote new sentence pairs, released under CC BY 4.0 with recorded consent. No personal or health data were collected, and we did not seek an institutional review determination.

9 Conclusion

Gain-only reporting would have credited the guard with its confirmation gain and stopped. The ledger adds that deletion alone explains part of it (+3.47 of +9.83 points), that scored passes can be fragments, and that a small pilot points to a usability cost that a larger audit with a frozen rule will test. The two bounded rows show the same pattern: the menu gains target readings and loses others, and the cue passed its development test but failed one clause of its fresh-item test. Human-written pairs and human-chosen keywords suggest the effects are not artifacts of authored items, but only people who use AAC can say whether any of it matters to them. The pre-specified study turns these offline findings into predictions that people can falsify. Analysis code, stored outputs

and scripts that rebuild every number and figure accompany the submission as anonymized supplementary material.

References

Agarwal, D.; Naaman, M.; and Vashistha, A. 2025. AI Suggestions Homogenize Writing Toward Western Styles and Diminish Cultural Nuances. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*. ACM.

Anonymous. 2027. Testing What an AAC Expander Is Allowed to Say: Input-Conditioned Commitment Contracts for LLM Sentence Expansion. Anonymized for review.

Arnold, K. C.; Chauncey, K.; and Gajos, K. Z. 2020. Predictive Text Encourages Predictable Writing. In *Proceedings of the 25th International Conference on Intelligent User Interfaces*. ACM.

Boruvka, A.; Almirall, D.; Witkiewitz, K.; and Murphy, S. A. 2018. Assessing Time-Varying Causal Effect Moderation in Mobile Health. *Journal of the American Statistical Association*, 113(523): 1112–1121.

Cai, S.; Venugopalan, S.; Seaver, K.; Xiao, X.; Tomanek, K.; Morris, M. R.; Kane, S.; Narayanan, A.; MacDonald, R. L.; Kornman, E.; Vance, D.; Casey, B.; Gleason, S. M.; Nelson, P. Q.; and Brenner, M. P. 2024. Using large language models to accelerate communication for eye gaze typing users with ALS. *Nature Communications*, 15: 9449.

Choudhury, S. H.; Kumar, A.; and Martin, L. J. 2025. Evaluating Human-LLM Representation Alignment: A Case Study on Affective Sentence Generation for Augmentative and Alternative Communication. In *Proceedings of the 14th International Joint Conference on Natural Language Processing and the 4th Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics*, 1618–1637. The Asian Federation of Natural Language Processing and The Association for Computational Linguistics.

Elgaar, M.; and Amiri, H. 2025. Linguistically-Controlled Paraphrase Generation. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, 20842–20864. Association for Computational Linguistics.

Frisch, B.; Wade, W.; Gaines, D.; Kinsella, M.; Peters, B.; Broderick, T.; and Vertanen, K. 2026. It's Complicated: On the Design and Evaluation of AI-Powered AAC Interfaces. Position paper, Speech AI for All workshop at CHI 2026, arXiv:2606.24854.

Gorenshtein, A.; Omar, M.; Jia, E.; Adiniaev, Y.; Daniel, O.; Kruskal, J.; Ahmed, M.; Brook, O.; Klang, E.; and Barash, Y. 2026. Intent Drift in LLM-Assisted BCI Communication: An In-Silico Benchmark Under Simulated Decoder Corruption. MedRxiv preprint, August 2026.

Gou, Z.; Shao, Z.; Gong, Y.; Shen, Y.; Yang, Y.; Duan, N.; and Chen, W. 2024. CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Critiquing. In *International Conference on Learning Representations*.

Hancock, J. T.; Naaman, M.; and Levy, K. 2020. AI-Mediated Communication: Definition, Research Agenda, and Ethical Considerations. *Journal of Computer-Mediated Communication*, 25(1): 89–100.

Hohenstein, J.; Kizilcec, R. F.; DiFranzo, D.; Aghajari, Z.; Mieczkowski, H.; Levy, K.; Naaman, M.; Hancock, J.; and Jung, M. F. 2023. Artificial intelligence in communication impacts language and social relationships. *Scientific Reports*, 13: 5487.

Jakesch, M.; Bhat, A.; Buschek, D.; Zalmanson, L.; and Naaman, M. 2023. Co-Writing with Opinionated Language Models Affects Users' Views. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*. ACM.

Klasnja, P.; Hekler, E. B.; Shiffman, S.; Boruvka, A.; Almirall, D.; Tewari, A.; and Murphy, S. A. 2015. Microrandomized Trials: An Experimental Design for Developing Just-in-Time Adaptive Interventions. *Health Psychology*, 34(Suppl.): 1220–1228.

Lacosse, E.; Tano, P.; and McNamee, D. C. 2026. Multi-System Cognitive Steering for Generative Communication. ArXiv preprint, version 1, October 2026, arXiv:2610.19177.

Madaan, A.; Tandon, N.; Gupta, P.; Hallinan, S.; Gao, L.; Wiegreffe, S.; Alon, U.; Dziri, N.; Prabhumoye, S.; Yang, Y.; Gupta, S.; Majumder, B. P.; Hermann, K.; Welleck, S.; Yazdanbakhsh, A.; and Clark, P. 2023. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems*, volume 36.

Mao, L.; Lee, J. H.; Faroqi-Shah, Y.; and Valencia, S. 2025. Design Probes for AI-Driven AAC: Addressing Complex Communication Needs in Aphasia. In *Proceedings of the 2025 ACM Designing Interactive Systems Conference*, 404–422. ACM.

Milton, D. E. M. 2012. On the Ontological Status of Autism: The 'Double Empathy Problem'. *Disability & Society*, 27(6): 883–887.

Nicolaidis, C.; Raymaker, D.; Kapp, S. K.; Baggs, A.; Ashkenazy, E.; McDonald, K.; Weiner, M.; Maslak, J.; Hunter, M.; and Joyce, A. 2019. The AASPIRE Practice-Based Guidelines for the Inclusion of Autistic Adults in Research as Co-Researchers and Study Participants. *Autism*, 23(8): 2007–2019.

Panickssery, A.; Bowman, S. R.; and Feng, S. 2024. LLM Evaluators Recognize and Favor Their Own Generations. In *Advances in Neural Information Processing Systems*, volume 37.

Pauli, A. B.; Augenstein, I.; and Assent, I. 2025. Mind the Style Gap: Meta-Evaluation of Style and Attribute Transfer Metrics. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, 21550–21564. Association for Computational Linguistics.

Qu, R.; Ungar, L.; and Sedoc, J. 2023. Conditioning on Dialog Acts improves Empathy Style Transfer. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, 13254–13271. Association for Computational Linguistics.

Shen, J.; Yang, B.; Dudley, J.; and Kristensson, P. O. 2022. KWickChat: A Multi-Turn Dialogue System for AAC Using Context-Aware Sentence Generation by Bag-of-Keywords. In *Proceedings of the 27th International Conference on Intelligent User Interfaces*, 853–867. ACM.

Valencia, S.; Cave, R.; Kallarackal, K.; Seaver, K.; Terry, M.; and Kane, S. K. 2023. "The less I type, the better": How AI Language Models can Enhance or Impede Communication for AAC Users. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*. ACM.

Valencia, S.; Huynh, J.; Jiang, E. Y.; Wu, Y.; Wan, T.; Zheng, Z.; Admoni, H.; Bigham, J. P.; and Pavel, A. 2024. COMPA: Using Conversation Context to Achieve Common Ground in AAC. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*. ACM.

Valencia, S.; Pavel, A.; Santa Maria, J.; Yu, S.; Bigham, J. P.; and Admoni, H. 2020. Conversational Agency in Augmentative and Alternative Communication. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems*. ACM.

Wang, R.; Sun, K.; and Guo, Y. 2026. Polite but Misaligned: Evaluating LLM Politeness Judgments Against Human Pragmatic Norms. In *Findings of the Association for Computational Linguistics: EMNLP 2026*, 26402–26421. Association for Computational Linguistics.

Wang, X.; Liu, Y.; Pan, J.; Ding, L.; Wang, L.; and Biemann, C. 2025. Chinese Toxic Language Mitigation via Sentiment Polarity Consistent Rewrites. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 35695–35711. Association for Computational Linguistics.

Williams-Ceci, S.; Jakesch, M.; Bhat, A.; Kadoma, K.; Zalmanson, L.; and Naaman, M. 2026. Biased AI writing assistants shift users' attitudes on societal issues. *Science Advances*, 12(11): eadw5578.

Xiang, W.; Xu, Y.; Fang, Y.; Teng, Z.; Jiang, Z.; Hu, B.; and Yang, J. 2025. SocializeChat: A GPT-Based AAC Tool Grounded in Personal Memories to Support Social Communication. ArXiv preprint, arXiv:2510.19017.

Zhao, H.; and Hawkins, R. D. 2025. Comparing human and LLM politeness strategies in free production. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 16188–16216. Association for Computational Linguistics.

Zheng, L.; Chiang, W.-L.; Sheng, Y.; Zhuang, S.; Wu, Z.; Zhuang, Y.; Lin, Z.; Li, Z.; Li, D.; Xing, E. P.; Zhang, H.; Gonzalez, J. E.; and Stoica, I. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems*, volume 36.

Ziegenbein, T.; Stahl, M.; and Wachsmuth, H. 2026. Teaching LLMs Human-Like Editing of Inappropriate Argumentation via Reinforcement Learning. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 38616–38637. Association for Computational Linguistics.

Zisk, A. H.; and Dalton, E. 2019. Augmentative and Alternative Communication for Speaking Autistic Adults: Overview and Recommendations. *Autism in Adulthood*, 1(2): 93–100.